Complete Binary Tree

- A **Complete Binary Tree** is a binary tree in which:
  - All levels are completely filled except possibly the last level.
  - The last level is filled from left to right.
- A complete binary tree is commonly used to implement a **Binary Heap**.
- It can be efficiently represented using an array.
- Height of a complete binary tree with `n` nodes is O(log n).

Array Representation of Complete Binary Tree

For a node at index `i` in a zero-based array:

- Left Child: `2 * i + 1`
- Right Child: `2 * i + 2`
- Parent: `(i - 1) / 2` (integer division, for `i > 0`)

Example:

```
    10
   /  \
  20   30
 / \
40  50
```

Array Representation:

`[10, 20, 30, 40, 50]`

- Root is stored at index 0.
- Left and right children are stored at calculated indexes.

SUNBEAM

- No gaps are present in the array representation.

Heap

- A **Heap** is a specialized complete binary tree that follows the Heap Property.
- It is commonly implemented using an array.
- It is mainly used to implement Priority Queues.
- The root node always contains the highest or lowest priority element, depending on the heap type.
- Heap is not a Binary Search Tree.

Types of Heap

1. Max Heap

- Every parent node has a value greater than or equal to its children.
- The root contains the maximum element.
- Every subtree also follows the Max Heap property.

Example:

```
    50
   /  \
  30   40
 / \
10  20
```

- Maximum element: Root (50).
- Parent is always greater than or equal to its children.

2. Min Heap

SUNBEAM

- Every parent node has a value smaller than or equal to its children.
- The root contains the minimum element.
- Every subtree also follows the Min Heap property.

Example:

```
    10
   /  \
  20   30
 / \
40  50
```

- Minimum element: Root (10).
- Parent is always smaller than or equal to its children.

Max Heap vs Min Heap

Feature	Max Heap	Min Heap
Root element	Maximum	Minimum
Parent-child property	Parent >= Children	Parent <= Children
Highest priority	Largest value	Smallest value
Common use	Maximum priority queue	Minimum priority queue

Heap Operations

1. Insertion in Heap

- A new element is always inserted at the next available position.

SUNBEAM

- This maintains the Complete Binary Tree property.
- After insertion, the Heap Property may be violated.
- **Heapify Up (Bubble Up)** is performed to restore the Heap Property.

Insertion Steps

1. Insert the new element at the end of the array.
2. Compare it with its parent.
3. If the Heap Property is violated, swap the element with its parent.
4. Repeat until the Heap Property is satisfied or the root is reached.

Example: Insertion in Max Heap

Initial Heap:

```
    50
   /  \
  30   40
 /
10
```

Insert 45:

- Insert 45 at the next available position.
- Compare 45 with its parent 30.
- Since 45 > 30, swap them.
- Compare 45 with its new parent 50.
- Since 45 < 50, stop.

Final Heap:

SUNBEAM

```
    50
   /  \
  45   40
 / \
10  30
```

- Time Complexity: O(log n)
- Space Complexity: O(1) auxiliary space for iterative insertion.

2. Deletion in Heap

- In a heap, deletion commonly means deleting the root element.
- In Max Heap, the maximum element is deleted.
- In Min Heap, the minimum element is deleted.
- **Heapify Down (Bubble Down)** is performed to restore the Heap Property.

Deletion Steps

1. Remove the root element.
2. Replace the root with the last element.
3. Remove the last position.
4. Compare the new root with its children.
5. In Max Heap, swap with the larger child if required.
6. In Min Heap, swap with the smaller child if required.
7. Repeat until the Heap Property is satisfied.

Example: Deletion in Max Heap

Initial Heap:

SUNBEAM

```
    50
   /  \
  30   40
 / \
10  20
```

Delete root 50:

- Replace root 50 with the last element 20.
- Remove the last position.
- Compare 20 with children 30 and 40.
- Swap 20 with the larger child 40.
- Heap Property is now satisfied.

Final Heap:

```
    40
   /  \
  30   20
 /
10
```

SUNBEAM

- Time Complexity: O(log n)
- Space Complexity: O(1) auxiliary space for iterative deletion.

3. Peek / Get Root

- Returns the root element without deleting it.
- Max Heap returns the maximum element.
- Min Heap returns the minimum element.
- Time Complexity: O(1).

4. Heapify

- Heapify is the process of restoring the Heap Property.
- Heapify Up is used after insertion.
- Heapify Down is used after root deletion or replacement.
- Building a heap from an unsorted array using bottom-up heap construction takes O(n) time.

Heap Operations - Complexity

Operation	Time Complexity
Get Root / Peek	O(1)
Insertion	O(log n)
Delete Root	O(log n)
Heapify Up	O(log n)
Heapify Down	O(log n)
Build Heap	O(n)
Search for an arbitrary element	O(n)

SUNBEAM

Priority Queue

- A **Priority Queue** is an abstract data type in which every element is associated with a priority.
- Elements are removed according to their priority, not necessarily their insertion order.
- The element with the highest priority is removed first.
- If two elements have equal priority, their removal order depends on the implementation.
- A heap is an efficient data structure for implementing a Priority Queue.

Types of Priority Queue

1. Min Priority Queue

- The element with the smallest priority value is removed first.
- Commonly implemented using a Min Heap.
- Example: Shortest Job First scheduling based on burst time.

2. Max Priority Queue

- The element with the largest priority value is removed first.
- Commonly implemented using a Max Heap.
- Example: Processing the highest-priority task first.

Priority Queue Implementation

A Priority Queue can be implemented using:

1. Unordered Array
2. Ordered Array
3. Unordered Linked List
4. Ordered Linked List
5. Binary Heap

SUNBEAM

1. Unordered Priority Queue

- Elements are stored without maintaining priority order.
- Insertion is performed at the end.
- Deletion requires searching for the highest-priority element.

2. Ordered Priority Queue

- Elements are stored in sorted order according to priority.
- Insertion requires finding the correct position.
- Deletion of the highest-priority element can be performed directly from one end.

Ordered vs Unordered Priority Queue

Operation	Unordered Array	Ordered Array
Insertion	O(1)	O(n)
Find highest priority	O(n)	O(1)
Delete highest priority	O(n)	O(1)
Peek highest priority	O(n)	O(1)

- In an unordered array, insertion is fast but deletion is slow.
- In an ordered array, insertion is slow but deletion is fast.
- Complexity assumes a fixed-size array with available capacity and priority order maintained during insertion.

SUNBEAM

Priority Queue Using Heap

- Binary Heap is commonly used to implement a Priority Queue.
- Max Heap implements Max Priority Queue.
- Min Heap implements Min Priority Queue.
- The root always contains the element with the highest priority according to the chosen heap ordering.

Operations

- **Insert:** Insert an element and perform Heapify Up.
- **Delete:** Remove the root and perform Heapify Down.

- **Peek:** Return the root without deletion.

Time Complexity

Operation	Heap-Based Priority Queue
Insertion	O(log n)
Delete Highest Priority	O(log n)
Peek Highest Priority	O(1)
Search Arbitrary Element	O(n)

Why Heap is More Efficient for Priority Queue?

- In an unordered array or linked list:
  - Insertion takes O(1).
  - Finding and deleting the highest-priority element takes O(n).
- In an ordered array or linked list:
  - Insertion takes O(n).
  - Finding and deleting the highest-priority element takes O(1), assuming deletion from the appropriate end.
- In a Binary Heap:
  - Insertion takes O(log n).
  - Deletion of the highest-priority element takes O(log n).
  - Peek takes O(1).
- Heap provides a good balance between insertion and deletion.
- It is particularly efficient when both insertion and deletion operations are frequent.

SUNBEAM

Comparison of Priority Queue Implementations

Implementation	Insertion	Delete Highest Priority	Peek
Unordered Array	O(1)	O(n)	O(n)
Ordered Array	O(n)	O(1)	O(1)
Unordered Linked List	O(1)	O(n)	O(n)
Ordered Linked List	O(n)	O(1)	O(1)
Binary Heap	O(log n)	O(log n)	O(1)

Applications of Heap and Priority Queue

- CPU Scheduling: Execute processes according to priority.
- Dijkstra's Algorithm: Select the vertex with the minimum tentative distance.
- Prim's Algorithm: Select the minimum-weight edge or next minimum-key vertex.
- Heap Sort: Sort elements using a heap.
- Event Scheduling: Process events based on their scheduled time.
- Job Scheduling: Execute jobs based on priority.
- Network Routing: Select routes based on cost or priority.
- K Largest / K Smallest Elements: Efficiently find the required elements.
- Merge K Sorted Lists: Select the smallest current element among multiple lists.
- Emergency Management: Process cases based on priority.

Java PriorityQueue

- Java provides the `PriorityQueue` class in the `java.util` package.
- By default, Java PriorityQueue works as a **Min Priority Queue**.

SUNBEAM

- The smallest element is removed first.
- It is internally implemented using a heap.
- It does not allow null elements.
- It does not guarantee sorted order while iterating over its elements.

Min Priority Queue

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

Max Priority Queue

```
PriorityQueue<Integer> pq =
    new PriorityQueue<>((a, b) -> b.compareTo(a));
```

Important Methods

Method	Description
add(e)	Inserts an element
offer(e)	Inserts an element
peek()	Returns root without removing
poll()	Removes and returns root
remove()	Removes root; throws exception if empty
size()	Returns number of elements
isEmpty()	Checks whether queue is empty

SUNBEAM

Important Java Points

- Default PriorityQueue is a Min Heap.
- `peek()` returns the smallest element by default.
- `poll()` removes and returns the smallest element by default.
- `add()` and `offer()` insert elements.
- `PriorityQueue` does not support efficient searching of arbitrary elements.
- Iterating over a PriorityQueue does not produce sorted order.
- To retrieve elements in priority order, repeatedly use `poll()`.

Important MCQ Points

- Complete Binary Tree fills the last level from left to right.
- Binary Heap is a Complete Binary Tree that follows the Heap Property.
- Max Heap root contains the maximum element.
- Min Heap root contains the minimum element.
- Heap is not a Binary Search Tree.
- In a Heap, siblings do not need to be sorted.
- Searching an arbitrary element in a heap takes O(n).
- Insertion in a heap takes O(log n).
- Deletion of the root takes O(log n).
- Peek operation takes O(1).
- Building a heap using bottom-up heap construction takes O(n).
- Max Heap is used for Max Priority Queue.
- Min Heap is used for Min Priority Queue.
- Java PriorityQueue is a Min Priority Queue by default.
- Java PriorityQueue does not guarantee sorted iteration.
- Heap is preferred when both insertion and priority-based deletion are frequent.
- Heap Sort has O(n log n) time complexity.

SUNBEAM